

程式碼大致流程

開場與環境設定 (Line 1 - 10)

「各位助教好，現在由我來為大家導讀我們 Level 2 虛擬電廠的 `scheduler.py` 核心架構。

首先在程式碼的最上方，我們宣告了三個關鍵的全域物理參數：充電效率 `ETA_CHG`、放電效率 `ETA_DIS`（皆設定為 0.95）以及電池老化成本 `BAT_AGING_COST`（設定為 300）。這三個參數是我們放寬『完美儲能假設』的基礎，後續會直接影響求解器的行為。」

模組一：MILP 引擎封裝 (`solve_vpp_milp` 函數, Line 30 - 150)

「為了實現動態重排程，我們將原本寫死在主程式裡的數學模型，重構成一個獨立的函數 `solve_vpp_milp`。這是一個可重複呼叫的『大腦引擎』。

這個引擎接收的參數非常特別，除了基本的預測資料，我們還加入了 `init_soc` (初始電量)、`init_gen_on` (機組狀態)、`gen_rem_up/down` (機組歷史債務)，以及 `init_u` 與 `fixed_load` (已預約任務快照)。這讓引擎具備了『時空接續能力』，它可以從任何一個 t 時刻完美接手過去的歷史，算出未來的最佳路徑。

在引擎內部的限制式中，我們做了幾個關鍵升級：

1. **供需平衡加入了背景負載**：方程式右邊加上了 `fixed_load.get(t, 0)`，用來保障已經預約的突發任務。
2. **電池能量守恆線性化**：為了避免除以 0.95 造成的浮點數崩潰，我們把這條公式改寫成了等式兩邊同乘 `ETA_DIS` 的線性化形式。
3. **違約金計算**：我們宣告了 `Shortfall` 變數來記錄未能滿足日前合約 `sell_da_commitment` 的短少電量，並在目標函數中乘上 `PENALTY_RATE` 進行巨額扣款。」

模組二：日前基準線建立 (Phase 1, Line 169 - 213)

「接下來進入主程式 `solve_vpp()`。

我們在 $t = 0$ 的時刻進行 Phase 1 的日前排程。這時系統會假設天氣是完美的（載入原始 `initial_pv_forecast`），也沒有合約包袱（`sell_da_commitment={}`）。

我們第一次呼叫 `solve_vpp_milp` 引擎，算出了一套 72 小時的完美計畫。如果成功，這套計畫中的售電量 `Sell[t].varValue` 就會被我們用一個字典 `Sell_DA` 鎖定起來，成為具備法律效力的『合約常數』，帶入下一個階段。」

模組三：動態模擬與氣候擾動注入 (Phase 2 準備, Line 219 - 236)

「正式進入 Phase 2 動態模擬前，我們為系統製造了一場危機。

我們建立了 `actual_pv_forecast` 字典，並在 $t = 30$ 到 $t = 35$ 這段中午尖峰時段，將太陽能的預測值強制乘以 0.60。這模擬了一場讓綠電暴跌 40% 的豪雨，準備考驗系統的應變能力。」

模組四：事件驅動與重排程 (Event Monitor & Rescheduling, Line 266 - 327)

「接著，系統進入 `for t in T:` 的逐時段推進迴圈。這裡是我們 Event-Triggered MPC 架構的靈魂。

每一小時的開頭，系統不會盲目重算，而是先啟動『安檢雷達』。系統會核對：『我原本預計要發的綠電 `expected_p`，有沒有大於現在真實氣候的上限 `actual_max_p`？』

如果有，雷達就會判定系統即將跳電，將 `needs_rescheduling` 設為 `True` 觸發重排程。

一旦觸發重排程：

1. 系統會立刻擷取『當下的歷史快照 (Snapshot)』，包含機組歷史連開時數 (`gen_rem_up_snap`) 以及還沒做完的任務 (`active_jobs_snap`)。
2. 最重要的是，系統會把過去已經答應接下的 `Sporadic` 任務，打包成 `fixed_load_snap`。
3. 接著，系統第二次呼叫 `solve_vpp_milp`，傳入這場大雨的真實預測、日前合約靶子、以及剛剛的歷史快照。
4. 引擎算出新的應變計畫後，我們會用 `.update()` 語法，把未來 (t 到 72 小時) 的變數無縫覆蓋掉，同時重建發電餘裕表 `margin_tracker`。」

模組五：雙層驗收測試與狀態推進 (Acceptance Test, Line 333 - 427)

「重排程確認安全後，系統提取當下的真實出力 `P_out`，準備迎接動態任務：

- **第一層攔截 (Sporadic Jobs)**：當硬死線任務抵達時，系統會利用『前瞻視野』去掃描未來的 `margin_tracker`。如果未來有足夠的空擋，系統就會『接受』並立刻把未來的餘裕扣除（預約保留），確保說到做到。
- **第二層消化 (Aperiodic Jobs)**：軟死線任務則會進入 `waiting_aperiodic` 佇列。系統會用當下剩餘的零星餘裕 `available_power_now`，採用貪婪法（能做多少算多少）來消化這些任務。